

Computer Programming 143 – Lecture 14

Arrays II

Faculty of Engineering



Copyright

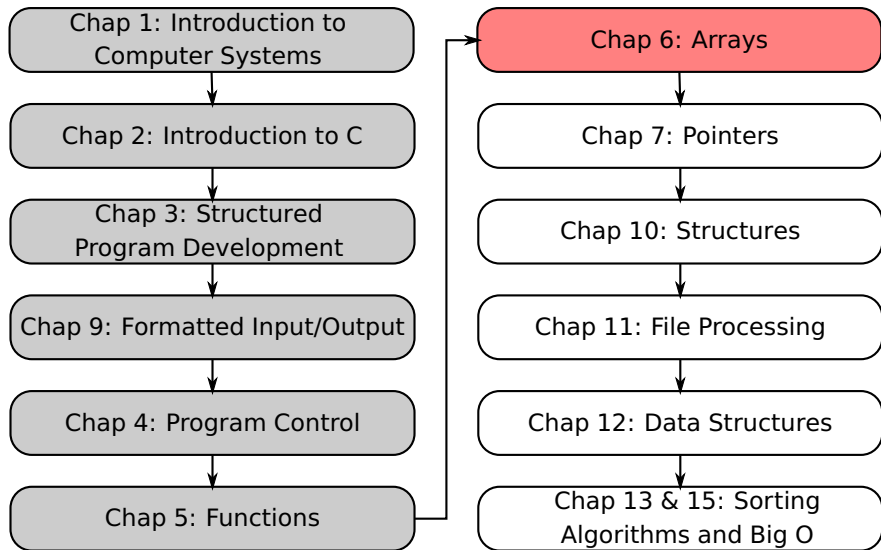
Copyright © 2026 Stellenbosch University
All rights reserved

Disclaimer

This content is provided without warranty or representation of any kind. The use of the content is entirely at your own risk and Stellenbosch University (SU) will have no liability directly or indirectly as a result of this content.

The content must not be assumed to provide complete coverage of the particular study material. Content may be removed or changed without notice.

Module Overview



Lecture Overview

- 1 Using Character Arrays to Store and Manipulate Strings (6.5)
- 2 Passing Arrays to Functions (6.7)
- 3 Searching Arrays: Linear and Binary Search (6.10)

6.5 Store and Manipulation of Strings

Character arrays

```
char str1[] = "first";
```

- String "first" is really a static array of characters
- Character arrays can be initialized using string literals
 - Null character `'\0'` terminates strings
 - `str1` actually has 6 elements
 - It is equivalent to

```
char str1[] = { 'f', 'i', 'r', 's', 't', '\0'};
```

- Can access individual characters
 - `str1[ 3 ]` is character `'s'`

6.5 Store and Manipulation of Strings

Character arrays (cont...)

- Array name is address of array, so **&** not needed for **scanf**
scanf("%s", string2);
 - Reads characters up until the first whitespace encountered (tab, newline, space)

6.5 Store and Manipulation of Strings

Problem

- Read a string (array of char) from the keyboard and combine with hard coded string. Also print the read string with spaces inserted between characters.

Pseudocode

*Declare a character (string1) array of 20 elements
(used for input, 20 assumed as maximum length)*

Initialise a character (string2) array with "string literal"

*Prompt the user for a string and read into array string1.
Print string1 and string2*

*For each character i in string1 up to the '\0' (end of string) character
print character i of string1 and a space*

6.5 Store and Manipulation of Strings

```
/* Treating character arrays as strings; Fig. 6.8 in Deitel & Deitel */
#include <stdio.h>
#define SIZE 20
int main(void) {
    char string1[SIZE]; // reserves SIZE characters
    char string2[] = "string literal"; // reserves 14+1 characters
    int i; // counter
    printf( "Enter a string (no longer than 19 characters): " );
    scanf( "%s", string1 ); // input ended by whitespace character
    // output strings
    printf( "string1 is: %s\nstring2 is: %s\n", string1, string2 );
    printf( "string1 with spaces between characters is:\n" );
    // output characters until null character is reached
    for ( i = 0; i < SIZE && string1[ i ] != '\0'; i++ ) {
        printf( "%c ", string1[ i ] );
    } // end for
    printf( "\n" );
    return 0; // indicates successful program termination
} // end main
```

6.5 Store and Manipulation of Strings

Output

```
Enter a string: Hello there
string1 is: Hello
string2 is: string literal
string1 with spaces between characters is:
H e l l o
```

6.7 Passing Arrays to Functions

Passing arrays

- To pass an array argument to a function, specify the name of the array without any brackets

```
int myArray[ 24 ];  
:  
myFunction( myArray, 24 );
```

- Array arguments passed call-by-reference
- Name of array is address of first element

6.7 Passing Arrays to Functions

Passing arrays (cont...)

- Function “knows” *where* the array is stored, but not array size
- Therefore, we usually pass the array size as a separate argument

Any changes modifies the data at original memory locations
because arrays are passed by reference

6.7 Passing Arrays to Functions

Function prototype

- Prototype for a function that takes an array as argument:

```
void myFunction( int b[], int arraySize );
```

- Specifies that the first argument of function **myFunction** is an array of integers. The second argument is the size of the array.
- To prevent the function from modifying the array, use the keyword **const**:

```
void myFunction( const int b[], int arraySize );
```

- See program listing in p. 318 in Deitel & Deitel

6.7 Passing Arrays to Functions

Passing array elements

- Passed by call-by-value
- Pass subscripted name (i.e., **myArray[3]**) to function
- Value of the element is copied into the parameter of the function
- Original element in array is unaffected by function

6.7 Passing Arrays to Functions

Example passing array elements (main C code)

```
// Passing arrays and individual array elements to functions.
#include <stdio.h>
#define SIZE 5
// function prototypes
void modifyArray(int b[], size_t size);
void modifyElement(int e);
int main(void) {
    int a[SIZE] = {0, 1, 2, 3, 4}; // initialize array a
    printf("Effects of passing entire array by reference:\n\nThe "
        "values of the original array are:");
    // output original array
    for (size_t i = 0; i < SIZE; ++i) {
        printf("%3d", a[i]);
    }
    printf("\n");
    modifyArray(a, SIZE); // pass array a to modifyArray by reference
}
```

6.7 Passing Arrays to Functions

Example passing array elements (main C code(vervolg...))

```
printf("The values of the modified array are:");  
for (size_t i = 0; i < SIZE; ++i) {  
    printf("%3d", a[i]);  
}  
  
printf("\n\nEffects of passing array element "  
"by value:\n\nThe value of a[3] is %d\n", a[3]);  
  
modifyElement(a[3]); // pass array element a[3] by value  
printf("The value of a[3] is %d\n", a[3]);  
return 0;  
}
```

6.7 Passing Arrays to Functions

Example passing array elements (cont...)

```
// in function modifyArray, "b" points to the original array "a" in memory
void modifyArray(int b[], size_t size) {
    // multiply each array element by 2
    for (size_t j = 0; j < size; ++j) {
        b[j] *= 2; // actually modifies original array
    }
}
```

```
// in function modifyElement, "e" is a local copy of array element
// a[3] passed from main
void modifyElement(int e) {
    e *= 2; // multiply parameter by 2
    printf("Value in modifyElement is %d\n", e);
}
```

6.10 Searching Arrays

Introduction

- When working with large amounts of data
- Search to see if it can match one of the array values to a **key value**
- Two searching techniques
 - Linear search
 - Binary search

6.10 Searching Arrays

Linear search (Search array for **key value**)

- Compare each element of array with key value
- Useful for small and unsorted arrays
- Assumes unique key values, e.g. student numbers
- Array does not have to be sorted

6.10 Searching Arrays

Example: linear search main C code

```
// fig06_14.c
// Linear search of an array.
#include <stdio.h>
#define SIZE 100
int linearSearch(const int array[], int key, size_t size);
int main(void) {
    int a[SIZE] = {0};
    // create some data
    for (size_t x = 0; x < SIZE; ++x) {
        a[x] = 2 * x;
    }
    printf("Enter integer search key: ");
    int searchKey = 0; // value to locate in array a
    scanf("%d", &searchKey);
```

6.10 Searching Arrays

Example: linear search main C code (cont...)

```
// attempt to locate searchKey in array a
int subscript = linearSearch(a, searchKey, SIZE);

if (subscript != -1) {
    printf("Found value at subscript %d\n", subscript);
}
else {
    puts("Value not found");
}
return 0;
}
```

6.10 Searching Arrays

Example: linear search (cont...)

*// compare key to every element of array until the location is found
// or until the end of array is reached; return subscript of element
// if key is found or -1 if key is not found*

```
int linearSearch(const int array[], int key, size_t size) {  
    for (size_t n = 0; n < size; ++n) {  
        if (array[n] == key) {  
            return n; // return location of key  
        }  
    }  
    return -1; // key not found  
}
```

6.10 Searching Arrays

Binary search

- Only for arrays **sorted** by key
- Compares **middle** element with **key**
 - If equal, match found
 - If **key** < **middle**, looks further in lower half of array
 - If **key** > **middle**, looks further in upper half of array
 - Repeat

6.10 Searching Arrays

Binary search (cont...)

- Very fast; at most n steps, where $2^n > \text{number of elements}$
 - A 30 element array takes at most 5 steps
 - $2^4 < 30 < 2^5$ so at most 5 steps
- More efficient than linear search

6.10 Binary Search

c[0]	1	← low = 0
c[1]	3	
c[2]	4	
c[3]	7	
c[4]	11	
c[5]	12	
c[6]	13	
c[7]	16	
c[8]	23	
c[9]	25	
c[10]	31	
c[11]	36	
c[12]	37	
c[13]	40	
c[14]	41	
c[15]	47	← high = 15

searchKey = 11

6.10 Binary Search

searchKey = 11

c[0]	1	← low = 0
c[1]	3	
c[2]	4	
c[3]	7	
c[4]	11	
c[5]	12	
c[6]	13	
c[7]	16	← middle = (high + low) / 2 = 7
c[8]	23	
c[9]	25	
c[10]	31	
c[11]	36	
c[12]	37	
c[13]	40	
c[14]	41	
c[15]	47	← high = 15

6.10 Binary Search

c[0]	1	← low = 0	searchKey = 11
c[1]	3		
c[2]	4		
c[3]	7		
c[4]	11		
c[5]	12		
c[6]	13		
c[7]	16	← middle = 7	
c[8]	23		searchKey < c[middle]
c[9]	25		
c[10]	31		
c[11]	36		
c[12]	37		
c[13]	40		
c[14]	41		
c[15]	47	← high = 15	

6.10 Binary Search

c[0]	1	← low = 0	searchKey = 11
c[1]	3		
c[2]	4		
c[3]	7		
c[4]	11		
c[5]	12		
c[6]	13	← high = middle - 1 = 6	
c[7]	16	← middle = 7	
c[8]	23		
c[9]	25		
c[10]	31		
c[11]	36		
c[12]	37		
c[13]	40		
c[14]	41		
c[15]	47		

6.10 Binary Search

c[0]	1	← low = 0	searchKey = 11
c[1]	3		
c[2]	4		
c[3]	7	← middle = (high + low) / 2 = 3	
c[4]	11		
c[5]	12		
c[6]	13	← high = 6	
c[7]	16		
c[8]	23		
c[9]	25		
c[10]	31		
c[11]	36		
c[12]	37		
c[13]	40		
c[14]	41		
c[15]	47		

6.10 Binary Search

c[0]	1	← low = 0	searchKey = 11
c[1]	3		
c[2]	4		
c[3]	7	← middle = 3	
c[4]	11		
c[5]	12		
c[6]	13	← high = 6	
c[7]	16		
c[8]	23		
c[9]	25		
c[10]	31		
c[11]	36		
c[12]	37		
c[13]	40		
c[14]	41		
c[15]	47		

searchKey > c[middle]

6.10 Binary Search

searchKey = 11

c[0]	1	
c[1]	3	
c[2]	4	
c[3]	7	← middle = 3
c[4]	11	← low = middle + 1 = 4
c[5]	12	
c[6]	13	← high = 6
c[7]	16	
c[8]	23	
c[9]	25	
c[10]	31	
c[11]	36	
c[12]	37	
c[13]	40	
c[14]	41	
c[15]	47	

6.10 Binary Search

searchKey = 11

c[0]	1	
c[1]	3	
c[2]	4	
c[3]	7	
c[4]	11	← low = 4
c[5]	12	← middle = (high + low) / 2 = 5
c[6]	13	← high = 6
c[7]	16	
c[8]	23	
c[9]	25	
c[10]	31	
c[11]	36	
c[12]	37	
c[13]	40	
c[14]	41	
c[15]	47	

6.10 Binary Search

c[0]	1
c[1]	3
c[2]	4
c[3]	7
c[4]	11
c[5]	12
c[6]	13
c[7]	16
c[8]	23
c[9]	25
c[10]	31
c[11]	36
c[12]	37
c[13]	40
c[14]	41
c[15]	47

searchKey = 11

← low = 4
← middle = 5
← high = 6

searchKey < c[middle]

6.10 Binary Search

c[0]	1
c[1]	3
c[2]	4
c[3]	7
c[4]	11
c[5]	12
c[6]	13
c[7]	16
c[8]	23
c[9]	25
c[10]	31
c[11]	36
c[12]	37
c[13]	40
c[14]	41
c[15]	47

searchKey = 11

$\text{high} = \text{middle} - 1 = 4$

$\text{low} = 4$

$\text{middle} = 5$

6.10 Binary Search

c[0]	1
c[1]	3
c[2]	4
c[3]	7
c[4]	11
c[5]	12
c[6]	13
c[7]	16
c[8]	23
c[9]	25
c[10]	31
c[11]	36
c[12]	37
c[13]	40
c[14]	41
c[15]	47

searchKey = 11

high = 4

low = 4

middle = (high + low) / 2 = 4

6.10 Binary Search

c[0]	1
c[1]	3
c[2]	4
c[3]	7
c[4]	11
c[5]	12
c[6]	13
c[7]	16
c[8]	23
c[9]	25
c[10]	31
c[11]	36
c[12]	37
c[13]	40
c[14]	41
c[15]	47

searchKey = 11

high = 4
low = 4
middle = 4

searchKey = c[middle]
return middle

Example: binary Search

```
/* function to perform binary search of an array */
int binarySearch( const int b[], int searchKey, int low, int high ) {
    int middle;

    while (low <= high) {

        middle = ( low + high ) / 2; /* get middle element*/

        if ( searchKey == b[ middle ] ) {
            return middle;
        } else if ( searchKey < b[ middle ] ) {
            high = middle - 1; /* search low end of array */
        } else {
            low = middle + 1; /* search high end of array */
        }
    }
    return -1; /* searchKey not found */
}
```

See Fig. 6.15 in Deitel & Deitel for an example of binary search

Today

Arrays II

- Character arrays
- Passing arrays to functions
- Searching arrays

Next lecture

Arrays III

- Sorting arrays

Homework

- 1 Study Sections 6.5, 6.7 and 6.10 in Deitel & Deitel
- 2 Do Self Review Exercise 6.2(e)
- 3 Do Exercises 6.6(a)-(f), 6.31